Databashantering



©Copyright SoftTech Development

Vad ska vi gå igenom?



- Vad är en databas?
- Vad är relationsdatabaser?
- Vad är dokumentdatabaser?

©Copyright SoftTech Development

Vad är en databas?

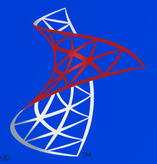
- En databas är organiserad lagring av data för att
  - Snabbt kunna hämta/läsa data
- Typiskt så lagras data på ett sådant sätt att
  - Duplicerat data undviks för att
    - Utnyttja lagringsytan på ett effektivt sätt
    - Reducera inkonsekvent data
- Tillhandahåller ett standardiserat frågespråk för åtkomst till data i databasen
- Tillhandahåller programvara för att kunna hantera databaser
  - DBMS (DataBase Management System)

©Copyright SoftTech Development

Typer av databaser













- Relationsdatabaser (Vårt fokus)
  - SQL Server, MySql, PostreSql, Oracle, IBM DB2, Sqlite osv…
- NoSql databaser
  - MongoDB, DynamoDB, CouchBase, CosmoDB, RedisJSON
- Hierarkiska databaser
  - MariaDB, IBM Information Management System (IMS)
- Kolumn databaser
  - Amazon Redshift, Snowflake, Google BigQuery, Apache Druid
- Med mera…

©Copyright SoftTech Development

Relationsdatabas

- En databas som består av tabeller som länkade(relaterade) till varandra och dess data
  - Därav namnet Relations Databas

CustomerId	First Name	Last Name	Address
1	Karl	Fredriksson	Kullavägen 2
2	Eva	Olsson	Strandvägen 4
3	Anna	Eriksson	Jullen 9
4	Bertil	Gustafsson	Citronvägen 18

OrderId	CustomerId	OrderDate
1001	1	2024-10-18
1002	1	2024-09-06
1003	2	2024-02-25
1004	3	2024-03-01

©Copyright SoftTech Development

Dokument Databaser (NoSQL)

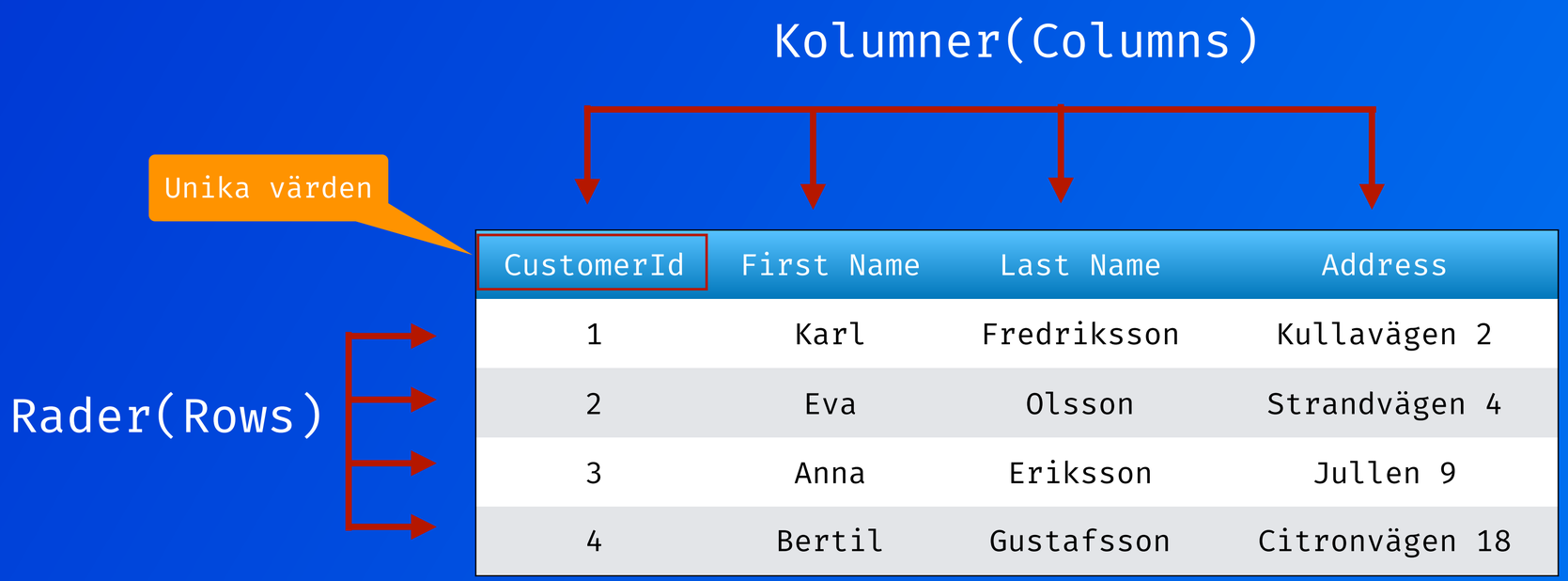
Lagras oftast i ett JSON liknande format kallat BSON(Binär JSON)
I NoSQL databaser är inte dupletter ett problem utan tvärtom så används dupletter för att öka hastigheten på hämtning av stora dokument

All data som behövs hämtas i ett svep.

```json
{
  "orderId": 1,
  "orderDate": "2024-10-01T13:12:33",
  "customer": {
    "customerId": 100,
    "addressLine": "Storgatan 8",
    "postalCode": "123 45",
    "city": "Storstaden",
    "firstName": "Anneli",
    "lastName": "Oskarsson",
    "phone": null
  }
}
```

©Copyright SoftTech Development

Grunderna i RDBMS

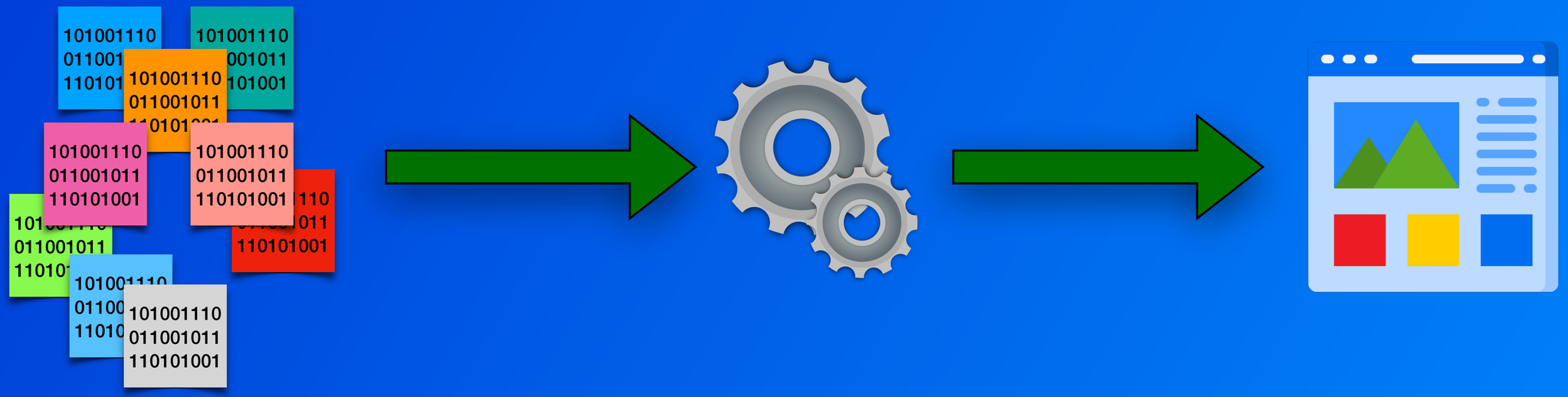


CustomerId	First Name	Last Name	Address
1	Karl	Fredriksson	Kullavägen 2
2	Eva	Olsson	Strandvägen 4
3	Anna	Eriksson	Jullen 9
4	Bertil	Gustafsson	Citronvägen 18

©Copyright SoftTech Development

Data kontra Information

- Data är obehandlat rådata
- Bearbetning av data tillhandahåller sammanhang för datat
- Bearbetat data är => Information



©Copyright SoftTech Development

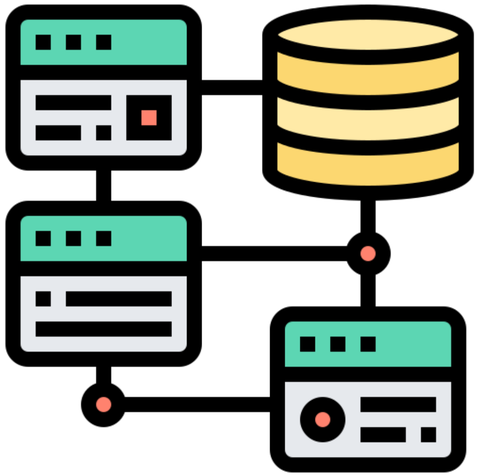
SQLite



- Vi kommer att använda SQLite som vår relationsdatabas i alla genomgångar och övningar
  - Anledningen är att SQLite (https://www.sqlite.org/) är en filbaserad databas och inte kräver någon installation av en databasmotor.
  - Används oftast under utveckling och EJ i produktion!
  - SQLite har lite underligheter för sig och är inte fullt kompatibel med standard SQL, men fungerar för att komma igång med en relationsdatabas
- Vi kommer att använda VSCode för manipulering av data och databaser.

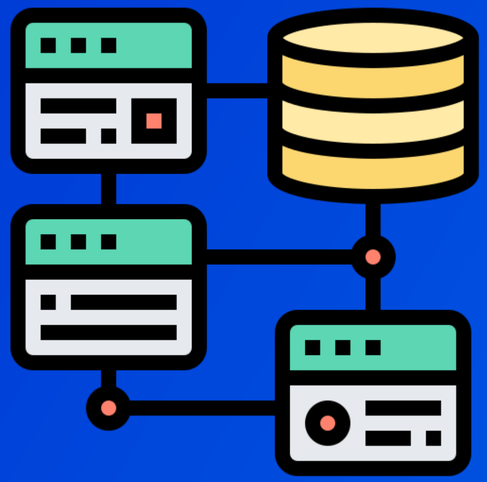
©Copyright SoftTech Development

Data Modellering



©Copyright SoftTech Development

Introduktion till data modellering



- Data modellering betyder att vi
  - Försöker att identifiera informations behovet i våra applikationer
    - Identifiera attribut(fält) och nycklar
    - Avgöra hur stor plats varje fält behöver
      - Hur många tecken behövs för e-post adresser?
  - Försöker att identifiera relationerna mellan vårt data
  - Normaliserar datat
    - Identifiera data som eventuellt kan förekomma flera gånger och eliminera sånt data

©Copyright SoftTech Development

Identifiera data(entities)

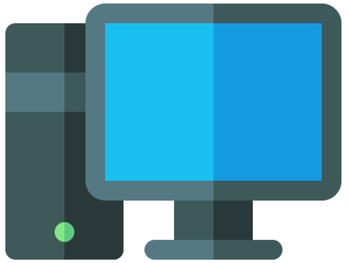
Exempel på entiteter(entities)



Person



Plats



Sak



Händelse

©Copyright SoftTech Development

Exempel (Customer)



- Vilken information behöver vi lagra för en kund?
  - Förnamn
  - Efternamn
  - E-Post
  - Telefon
  - Adress
  - Postnummer
  - Postort

©Copyright SoftTech Development

Tabeller



- Nu när vi vet vilken information som behövs kan vi designa tabellen Customer
  - Varje egenskap som vi hittat ska nu omvandlas till ett tabell fält.
  - Varje fält ska ha en datatyp som anger utrymmet som behövs för att lagra fältets värde vare sig mer eller mindre
  - Anledningen är att vi vill på ett så effektivt sätt som möjligt utnyttja diskutrymmet för databasen

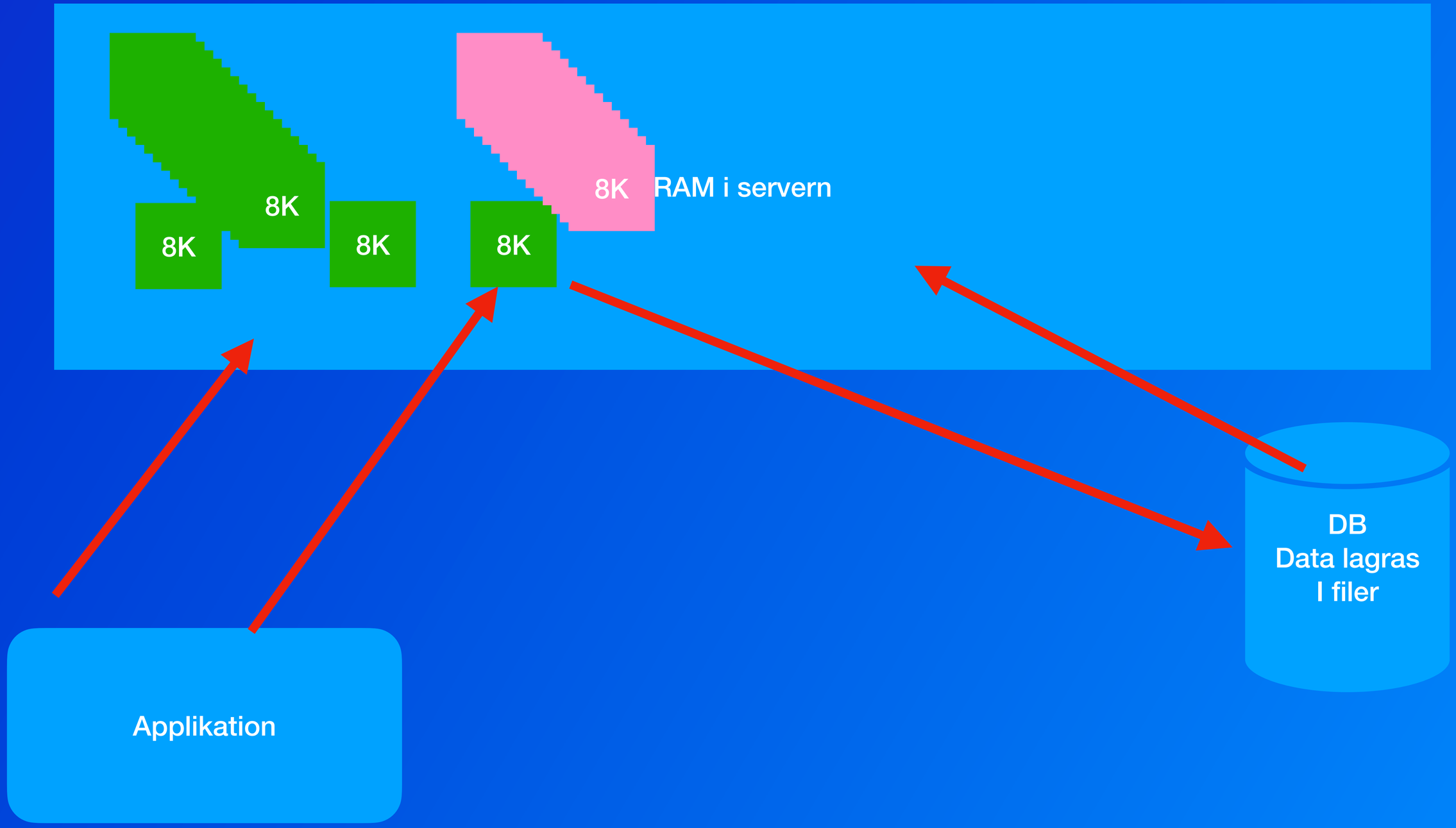
©Copyright SoftTech Development

Tabell design



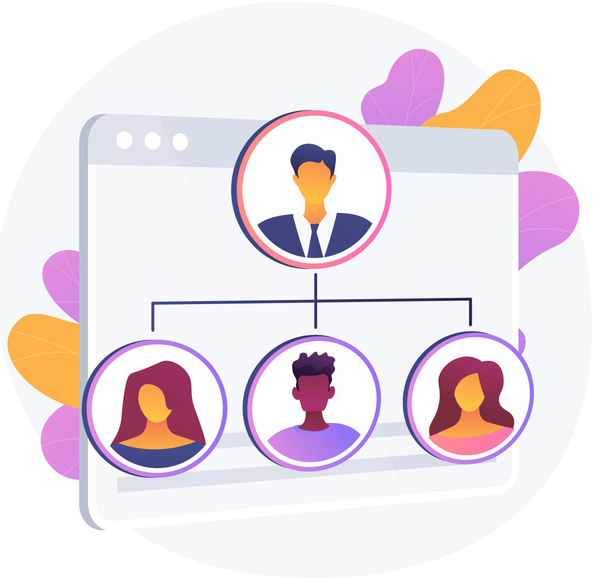
Field	DataType	Constraint
FirstName	TEXT	
LastName	TEXT	
Email	TEXT	
Phone	TEXT	
Address	TEXT	
PostalCode	TEXT	
City	TEXT	

©Copyright SoftTech Development



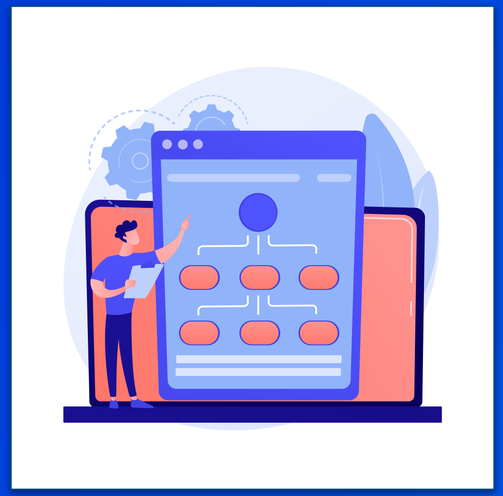
©Copyright SoftTech Development

ERD



©Copyright SoftTech Development

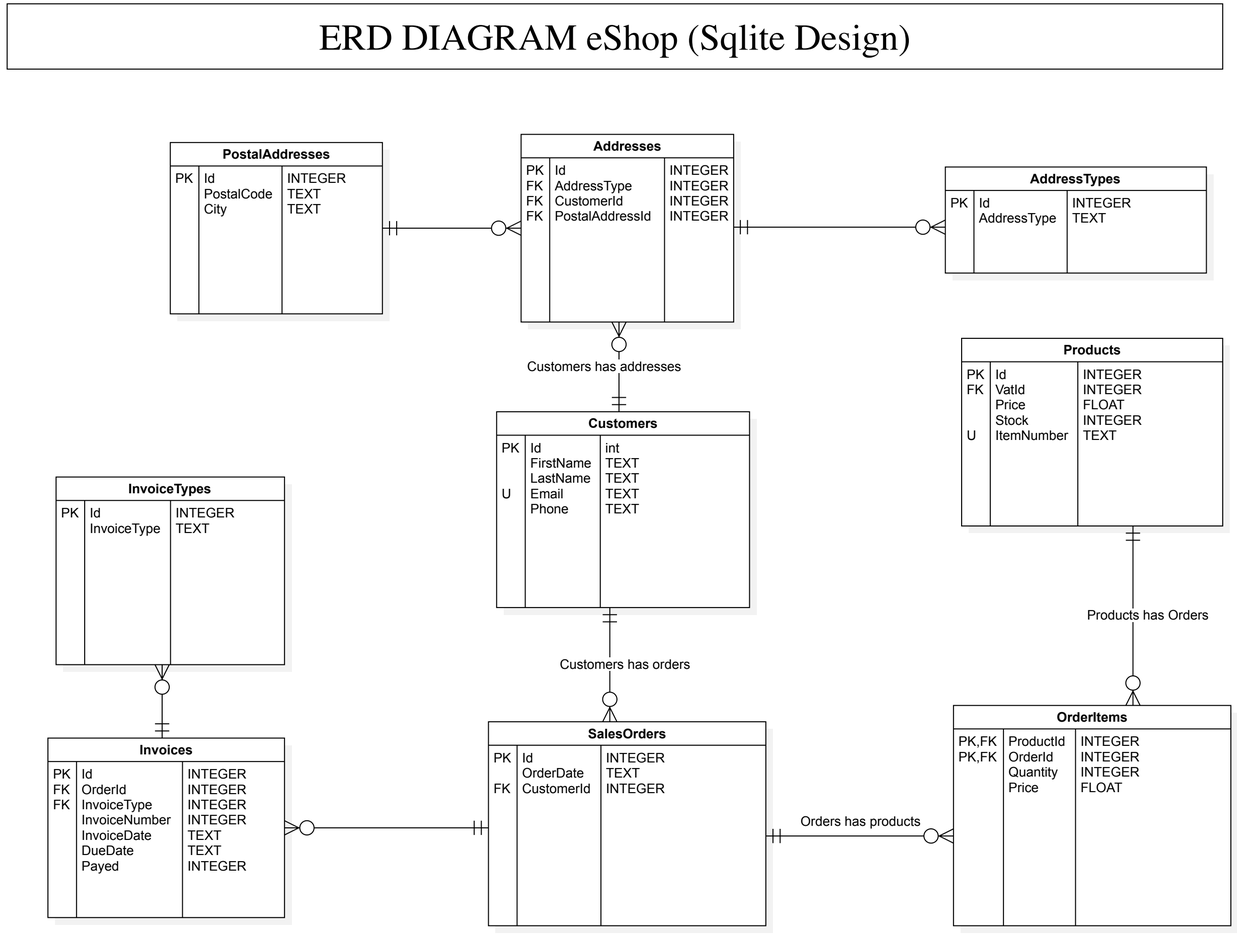
Entity Relationship Diagram



- Entity Relationship Diagram eller ERD
  - Är ett sätt att visualisera sin databas design
  - Mycket likt hur vi använder UML för att visualisera vår applikations design med klasser och objekt
- Syftet med ett ERD diagram är att:
  - Definiera vilka tabeller som system består av
  - Vilken information varje tabell ska hantera
  - Vilka datatyper varje fält ska hantera
  - Vilka regler som ska finnas i tabellen och fälten
- Dessutom blir det en dokumentation över databasen
  - Som kan användas vid diskussioner

©Copyright SoftTech Development

Exempel



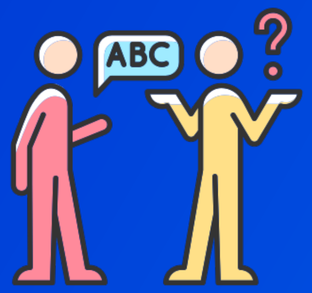
©Copyright SoftTech Development

Frågespråket SQL



©Copyright SoftTech Development

Dialekter av SQL



- SQL (Structured Query Language) finns i olika dialekter detta beror på att olika leverantörer av databasmotorer(DBMS) har valt olika sätt att implementera SQL standarden ANSI-SQL
- T ex så har vi
  - TS-SQL som är Microsoft implementering av ANSI-SQL
  - PL-SQL som är Oracle och IBM's variant av ANSI-SQL
- Andra som följer ANSI-SQL är bland annat
  - MySql
  - PostgreSQL

©Copyright SoftTech Development

SQL Kategorier



- SQL språket är uppdelat i 3 kategorier
  - DML (Data Manipulation Language)
    - För att manipulera data
  - DDL (Data Definition Language)
    - För att skapa och manipulera databas objekt
  - DCL (Data Control Language)
    - För att hantera behörighet till olika objekt i en databas eller databas servern

©Copyright SoftTech Development

DDL



- Syntax:
  - Skapa objekt

```
CREATE DATABASE databasnamn;
```

```
CREATE TABLE tabellnamn(col1 datatyp, col2 datatyp,…);
```

  - Ändra objekt

```
ALTER TABLE tabellnamn ADD Column_Name datatype;
```

  - Ta bort objekt

```
DROP TABLE tabellnamn;
DROP DATABASE databasnamn;
```

©Copyright SoftTech Development

DDL Exempel 1:



- Skapa en tabell

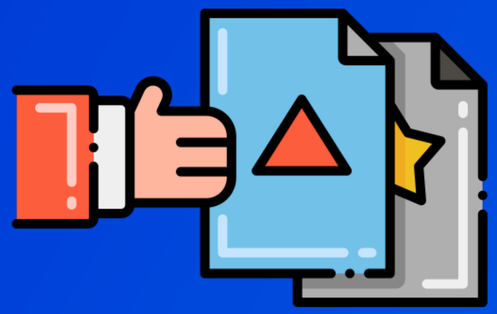
```
1  CREATE TABLE Customers(
2      Id INTEGER PRIMARY KEY AUTOINCREMENT,
3      FirstName VARCHAR(20),
4      LastName VARCHAR(30),
5      Email VARCHAR(128),
6      Phone VARCHAR(20)
7  );
```

PRIMARY KEY:
Garanterar att ingen annan rad i tabellen kan ha samma värde.

AUTOINCREMENT:
Vi behöver inte ange ett värde i kolumnen istället skapas ett nytt värde för varje ny rad.

©Copyright SoftTech Development

DDL Exempel 2:



- Ta bort en tabell

```
1  DROP TABLE Customers;
```

- Uppdatera en tabell
  - Lägg till en ny kolumn

```
1  ALTER TABLE Customers
2      ADD COLUMN AddressLine VARCHAR(60);
```

©Copyright SoftTech Development

Uppgift



Vi ska nu skapa våra tabeller för RunStore

©Copyright SoftTech Development

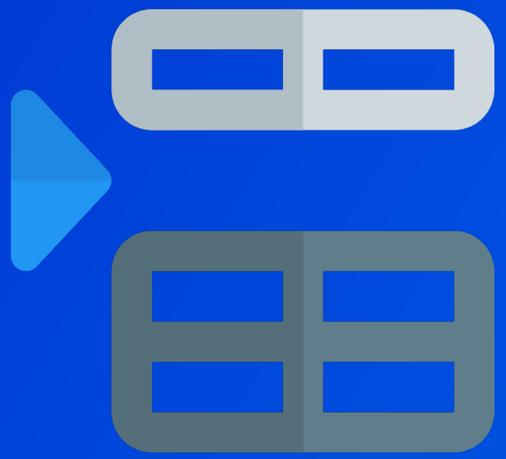
DML



- DML används för att utföra CRUD operationer på data
  - CREATE(INSERT)
  - READ(SELECT)
  - UPDATE
  - DELETE

©Copyright SoftTech Development

CREATE(INSERT)

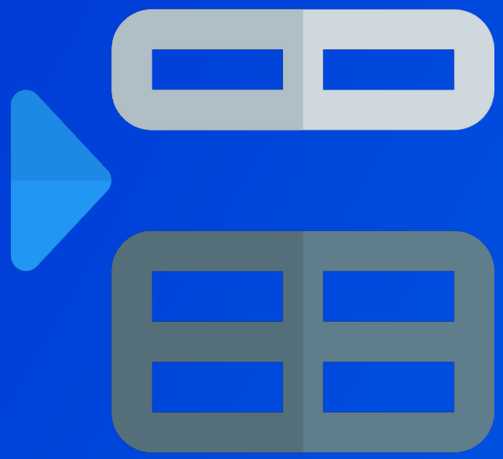


- Syntax

```
INSERT TABLE Customers(
   Columnnamn1,
   Columnnamn2,
   Columnnamn3,
   …
)
VALUES(
   Värde1,
   Värde2,
   Värde3,
   …
);
```

©Copyright SoftTech Development

INSERT Exempel



```sql
INSERT INTO Customers(
    FirstName,
    LastName,
    Email,
    Phone)
VALUES(
    'Michael',
    'Gustavsson',
    'michael@gmail.com',
    '070-4455667'
);
```

©Copyright SoftTech Development

READ(SELECT)



- Syntax:

```
SELECT * FROM TABLE;
Returnerar alla rader och alla kolumner ifrån tabellen
```

```
SELECT Column1, Column2, … FROM TABLE;
Returnerar alla rader och endast de kolumner som är
angivna i SELECT listan
```

```
SELECT Column1, Column2, … FROM TABLE WHERE Condition;
Returnerar endast raderna som uppfyller villkoret och
endast de kolumner som är angivna i SELECT listan
```

©Copyright SoftTech Development

SELECT Exempel



```sql
SELECT * FROM Customers;
```

```sql
SELECT FirstName, LastName FROM Customers;
```

```sql
SELECT FirstName, LastName FROM Customers
WHERE FirstName = 'Michael';
```

©Copyright SoftTech Development

UPDATE



- Syntax:

```
UPDATE tablename SET Column1 = value, Column2 = value;
Uppdaterar kolumnerna i SET listan och alla rader.
```

```
UPDATE tablename SET Column1 = value, Column2 = value
WHERE Condition;
Uppdaterar kolumnerna i SET listan och endast de rader
som matchar villkoret.
```

©Copyright SoftTech Development

UPDATE Exempel



```
1  UPDATE Customers SET Phone = '111-22222';
```

Uppdaterar alla rader i tabellen till samma telefonnummer⚠️

```
1  UPDATE Customers SET Phone = '111-22222'
2  WHERE Id = 1;
```

Uppdaterar bara rader i tabellen som har id 1

©Copyright SoftTech Development

DELETE



- Syntax:

```
DELETE FROM tablename;
Raderar alla rader i tabellen
```

```
DELETE FROM tablename WHERE Condition;
Raderar endast de rader som matchar villkoret.
```

©Copyright SoftTech Development

DELETE Exempel



```sql
DELETE FROM Customers;
```

```sql
DELETE FROM Customers WHERE Id = 2;
```

©Copyright SoftTech Development

Uppgift



Vi ska nu titta på hur vi kan manipulera data i våra nya tabeller

©Copyright SoftTech Development

Constraints



©Copyright SoftTech Development

Constraints(Regler)



- Constraints är egenskaper(attribut) som vi kan sätta på kolumner men även på tabeller.
- Kan t ex vara
  - Regler som garanterar att en kolumn alltid är unik
  - Regler som definierar förhållanden mellan tabeller
- Vi kan sätta regler när vi skapar tabellerna eller efteråt

©Copyright SoftTech Development

Typ av regler



- Vanliga regler som används i tabeller:
  - NOT NULL - Kolumner kan inte vara utan värde
  - UNIQUE - Kolumnens värde måste vara unikt för varje rad
  - PRIMARY KEY - NOT NULL och UNIQUE kombinerat
  - FOREIGN KEY - Till för att skydda relationer mellan tabeller
  - CHECK - Garanterar att värden i en kolumn uppfyller vissa krav
  - DEFAULT - Anger ett standardvärde om inget värde ges för kolumnen
  - CREATE INDEX - Är till för att specificera regler för att så snabbt som möjlig kunna hämta data ur en tabell

©Copyright SoftTech Development

Constraints Exempel



Anger regler vid skapandet av tabellen
För SQLite är detta det rekommenderade sättet !

```sql
CREATE TABLE Customers(
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName VARCHAR(20) NOT NULL,
    LastName VARCHAR(30) NOT NULL ,
    Age INTEGER NOT NULL,
    Email VARCHAR(128) NOT NULL UNIQUE ,
    Phone VARCHAR(20),
    CHECK(Age >= 18)
);
```

©Copyright SoftTech Development

Constraints Exempel



PRIMARY KEY
Detta är ANSI-SQL och fungerar ej för SQLite

```sql
ALTER TABLE Customers
ADD CONSTRAINT PK_Customer PRIMARY KEY(Id);
```

NOT NULL
Detta är ANSI-SQL och fungerar ej för SQLite

```sql
ALTER TABLE Customers
ALTER COLUMN FirstName VARCHAR(20) NOT NULL;
```

©Copyright SoftTech Development

Constraints Exempel



UNIQUE
Detta är ANSI-SQL och fungerar ej för SQLite

```
1  ALTER TABLE Customers
2  ADD CONSTRAINT UC_Email UNIQUE(Email);
```

CHECK
Detta är ANSI-SQL och fungerar ej för SQLite

```
1  ALTER TABLE Customers
2  ADD CONSTRAINT CHK_Age CHECK (Age>=18);
```

©Copyright SoftTech Development

Constraints Exempel



DEFAULT
Detta är ANSI-SQL och fungerar ej för SQLite

```
1  ALTER TABLE Customers
2  ALTER Phone SET DEFAULT '111-111111';
```

©Copyright SoftTech Development

Uppgift



Vi ska nu göra om våra tabeller
så vi skyddar data med constraints

©Copyright SoftTech Development